

Computer Science 341 – Lab 5 - Oh, What Could *Possibly* Go Wrong? 🤔

Assigned: 10/7/25

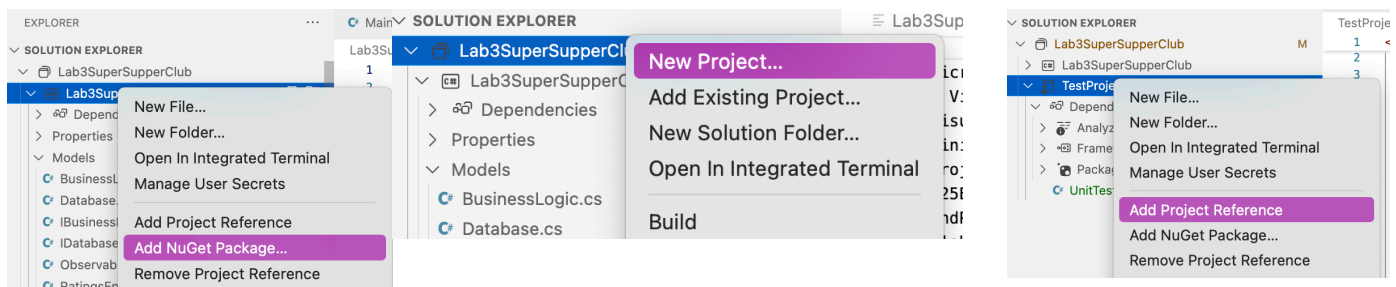
Due: 10/14/25, 1:20 PM

Objective: Learn how to construct and run unit tests in Visual Studio; encourage reading the documentation and independent learning.

Overview: Developers *love* doing unit tests, and so should be at the heart of software development, not an afterthought. In this lab, you will perform basic unit testing, to verify that the app's basic functionality is there.

Getting Started:

1. Study [this tutorial on unit testing](#) to see how easy it is to do unit testing in VS Code.
2. Create a new Console-based application, **Lab5-FML** (replace # with your initials).
3. Add the model and database classes from lab 3, i.e., exclude the UI components.
4. Add the Supabase NuGet package to your project.
5. Add a new project to your solution, an xUnit Test Project, call it **TestProject1**
6. Add a project reference to your Lab5-FML application.
7. Get ready to use using Lab3SuperSupperClub.Models; and the like in the testing project.



Testing, Testing:

Now that you have a testing project set up, it's time to put it to the test! Write unit tests to verify the public methods in the `IBusinessLogic` interface, specifically that a user can:

1. add a new supper club to the database
2. delete a supper club in the database
3. edit a supper club in the database
4. delete all supper clubs in the database

Each test should be structured into Arrange, Act, and Assert sections.

In the Arrange section, create whatever variables you need (e.g., a `BusinessLogic` instance (storing it in an `IBusinessLogic` variable)).

In the Act section, you will do something -- add / update / delete / retrieve / delete all

In the Assert section you will test if your activity worked. For example, you might add a new supper club in act; then in assert, fetch that supper club and verify that it was present in the database.

```
using Xunit;

namespace TestProject1
{
    public class UnitTest1
    {
        [Fact]
        public void Test1()
        {
            // Arrange
            int a = 5;
            int b = 10;

            // Act
            int result = a + b;

            // Assert
            Assert.Equal(15, result);
        }
    }
}
```

Be sure to test edge cases (supper clubs with invalid fields; deleting with an invalid id; etc). Be as thorough as you can - negative rating, nonexistent supper club, etc.

Name your tests appropriately, so it is clear what they are doing. For example TestSuccessfulSCAddition(); TestAdditionNegativeRating(); TestAdditionDuplicateSCId(); etc.

Testing should produce all green checkmarks, plan your assertions accordingly.

You may need to add additional methods to facilitate testing, but the IBusinessLogic should never have direct access to the Database.

You may use both [Fact] and [Theory] as necessary.

All testing must be done through the business logic layer -- do not access the Database directly.

Submission:

Zip up your entire (cleaned) project folder and upload the zipped file to Canvas by the specified deadline.

Include as a separate unzipped image a screenshot showing the Testing Window.